Unsupervised Learning, Recommenders, Reinforcement Learning
Coursera Course 3/3 Notes
Nicolas Rebollo Canedo-Arguelles
Started Oct 26, 2026

Table of Contents

1. Week 1 Notes – Unsupervised Learning..... 3
1.1. What is clustering?..... 3
1.2. K-means intuition..... 3
1.3. K-means algorithm..... 3
1.4. Optimization objective..... 3
1.5. Initializing K-means..... 4
1.6. Choosing the number of clusters..... 5
1.7. Finding unusual events..... 5
1.8. Gaussian (normal) distribution..... 6
1.9. Anomaly detection algorithm..... 7
1.10. Developing and evaluating an anomaly detection system..... 7
1.11. Anomaly detection vs. supervised learning..... 8
1.12. Choosing what features to use..... 9
2. Week 2 Notes – Recommender Systems..... 9
2.1. Making recommendations..... 9
2.2. Using per-item features..... 10
2.3. Collaborative filtering algorithm..... 10
2.4. Binary labels: favs, likes, and clicks..... 11
2.5. Mean normalization..... 11
2.6. TensorFlow implementation of collaborative filtering..... 12
2.7. Finding related items..... 12
2.8. Collaborative filtering vs. content-based filtering..... 13
2.9. Deep learning for content-based filtering..... 13
2.10. Recommending from a large catalogue..... 14
2.11. Ethical use of recommender systems..... 14
2.12. TensorFlow implementation of content-based filtering..... 15
2.13. Reducing the number of features..... 15
2.14. PCA algorithm..... 15
2.15. PCA in code..... 16
3. Week 3 Notes – Reinforcement Learning..... 17
3.1. What is reinforcement learning?..... 17
3.2. Mars rover example..... 17

3.3. The return in reinforcement learning..... 17
3.4. Making decisions: policies in reinforcement learning..... 18
3.5. Review of key concepts..... 18
3.6. State-action value function definition..... 18
3.7. State-action value function example..... 19
3.8. Bellman equation..... 19
3.9. Random (stochastic) environment..... 20
3.10. Example of continuous state space applications..... 20
3.11. Lunar lander..... 20
3.12. Learning the state-value function..... 20
3.13. Algorithm refinement: improved neural network architecture..... 21
3.14. Algorithm refinement: ε-greedy policy..... 22
3.15. Algorithm refinement: mini-batch and soft updates..... 22
3.16. The state of reinforcement learning..... 23

1. Week 1 Notes – Unsupervised Learning

1.1. What is clustering?

- Finds data points that are related to each other
- Supervised learning has a dataset which includes the target outputs, y, while unsupervised learning has a dataset with only features and no target outputs.
- Clustering looks for groups/clusters of points that are similar to each other

1.2. K-means intuition

- First takes a random guess at where the center of each cluster is
- K-means does 2 things repeatedly: assign points to cluster centroids, and move cluster centroids
- Center of a cluster is called cluster centroid
- For each point in the dataset, it finds the closest centroid, and assign each point to its closest centroid (Step 1)
- Looks at all points for a certain cluster, takes the average location of the points, and moves the previous centroid guess to this new average location (Step 2)
- Repeat until centroid location converges to a single location

1.3. K-means algorithm

- Randomly initialize K cluster centroids, $\mu_1, \mu_2, \ldots, \mu_K$
- Repeat {

Assign points to cluster centroids
for i = 1 to m
$c^{(i)}$:= index (from 1 to K) of cluster centroid closest to $x^{(i)}$

$$min_k \| x^{(i)} - \mu_k \|^2$$

Move cluster centroids
for k = 1 to K
μ_k := average (mean) of points assigned to cluster k

$$\mu_1 = \frac{1}{4}\left[x^{(1)} + x^{(5)} + x^{(6)} + x^{(10)}\right]$$

}

- If a cluster has 0 points assigned to it, eliminate the cluster to avoid taking the average of nothing. Or you can randomly re-initialize the cluster (less common)

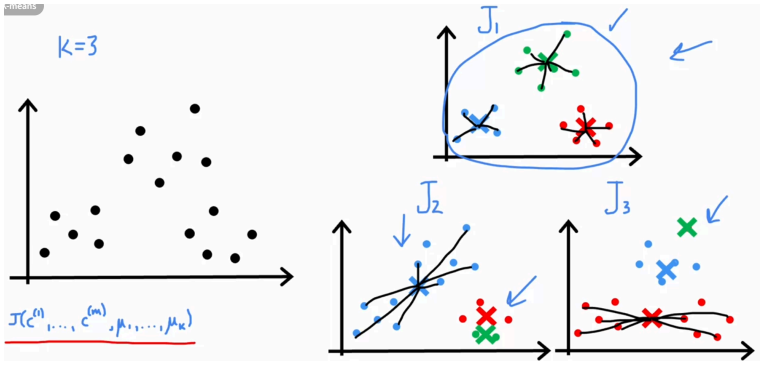
1.4. Optimization objective

- K-means algorithm also optimizes a cost function (but doesn't use gradient descent, like supervised learning)
- $c^{(i)}$ = index of cluster (1,2,...,K) to which example $x^{(i)}$ is currently assigned

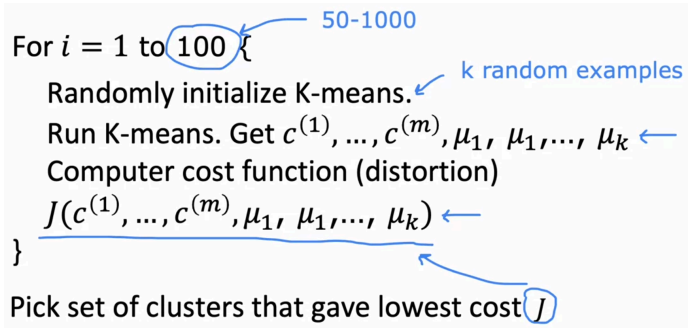
- μ_k = cluster centroid k
- $\mu_{c^{(i)}}$ = cluster centroid of cluster to which example $x^{(i)}$ has been assigned
- Cost function: $J(c^{(1)}, \ldots, c^{(m)}, \mu_1, \ldots, \mu_K) = \frac{1}{m}\sum_{i=1}^{m} ||x^{(i)} - \mu_{c^{(i)}}||^2$
    - It is the average squared distance between every training example and its corresponding cluster
    - This is called the *distortion function*
- It is guaranteed to converge, the distortion function should go down or stay the same. If it ever goes up, there is a bug in the code. If it ever stops going down, it usually means K-means has converged, and you can stop running the algorithm.

1.5. Initializing K-means

- The first step was to choose random locations for cluster centroids, but how do we take that random guess?
- Choose $K < m$ (there should always be less clusters than data points)
- Randomly pick K training examples and set $\mu_1, \ldots, \mu_k$ equal to these K examples
- K-means can get stuck in a local minima with a poor random initialization

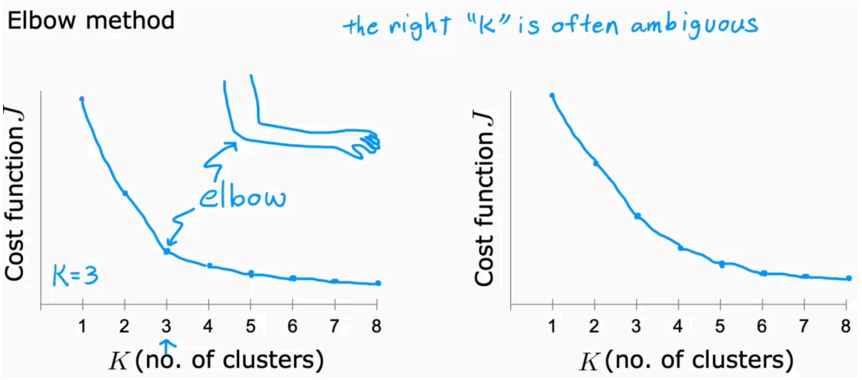


- Can run K-means multiple times and find the best initialization. Computer cost function for all solutions and pick the solution with the lowest cost.
- This multiple K-means random initialization algorithm provides better clustering:

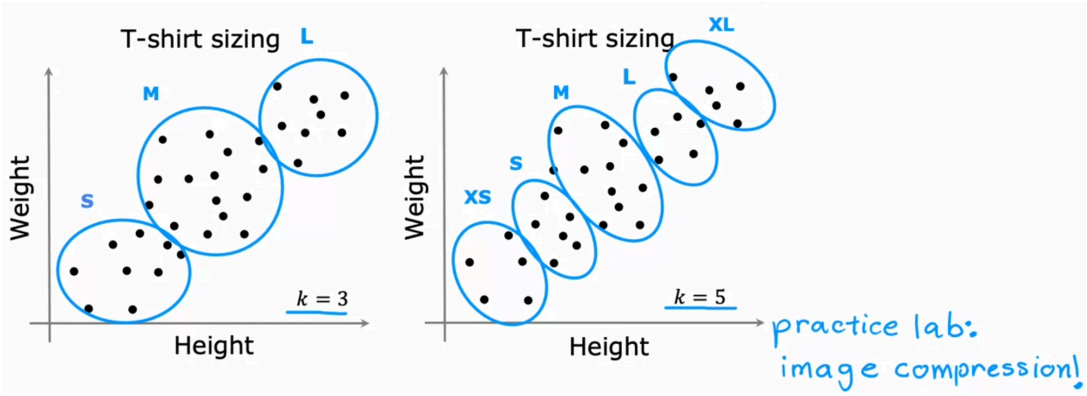


1.6. Choosing the number of clusters

- For a lot of cases, the number of cases is ambiguous
- Can use the elbow method:

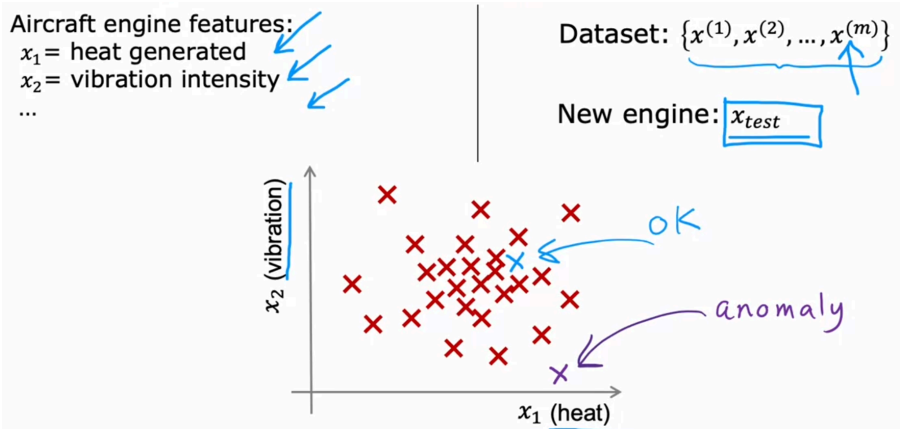


- One technique that DOES NOT work is to choose the k that minimizes J, because more k usually always minimizes J.
- Often, you get clusters and do something with them. Thus, evaluate K-means based on how well it performs on that later purpose.

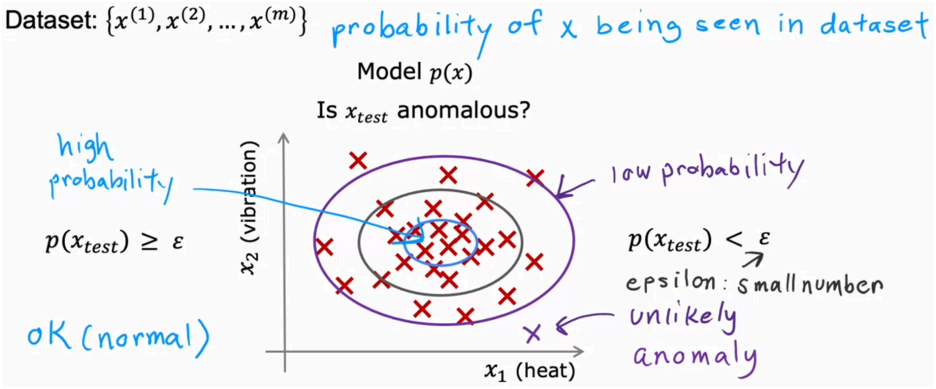


1.7. Finding unusual events

- Anomaly detection algorithms look at unlabeled datasets of normal events and learn to raise a flag for anomalous events

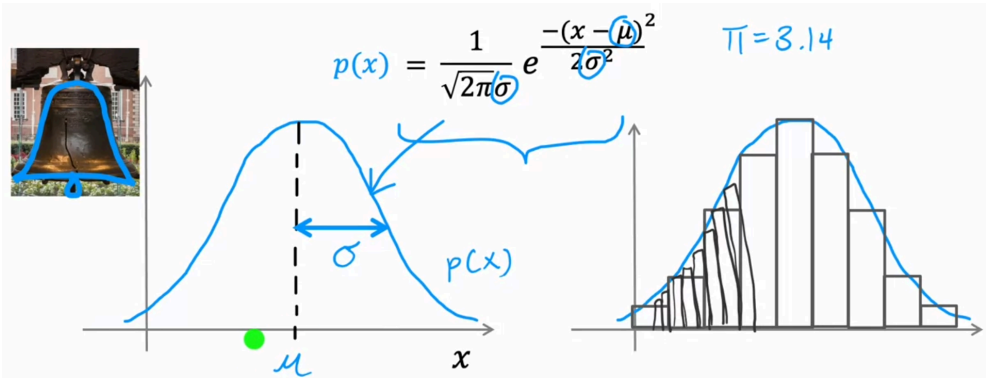


- Density estimation is the most common technique for anomaly detection

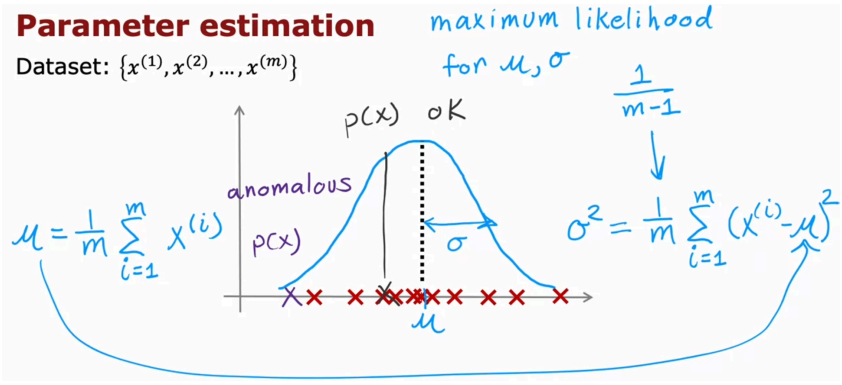


1.8. Gaussian (normal) distribution

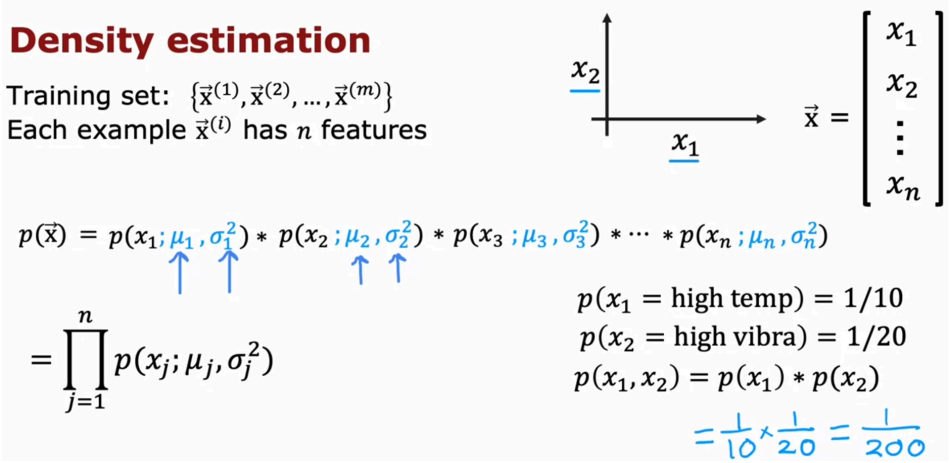
- x is a number, and probability of x is determined by a Gaussian with mean μ, variance σ^2



- Changing μ, or the mean, shifts the curve left or right
- Changing σ, or the standard deviation, makes it wider or thinner



1.9. Anomaly detection algorithm



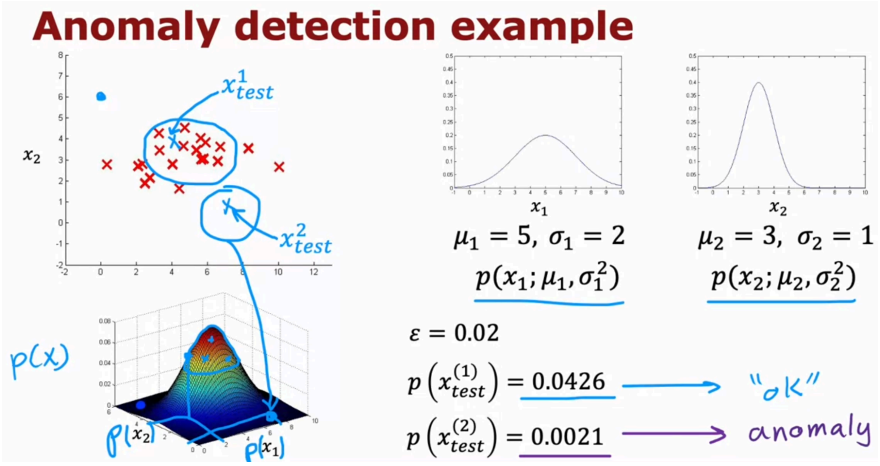
1. Choose n features x_i that you think might be indicative of anomalous examples
2. Fit parameters $\mu_1, \ldots, \mu_n, \sigma_1^2, \ldots, \sigma_n^2$

   $\mu_j = \frac{1}{m}\sum_{i=1}^{m} x_j^{(i)}$ $\qquad$ $\sigma_j^2 = \frac{1}{m}\sum_{i=1}^{m} (x_j^{(i)} - \mu_j)^2$

   Or, in vectorized form:

   $\vec{\mu} = \frac{1}{m}\sum_{i=1}^{m} \vec{x}^{(i)}$ $\qquad$ $\vec{\mu} = [\mu_1, \mu_2, \ldots, \mu_n]$
3. Given new example x, compute $p(x)$:

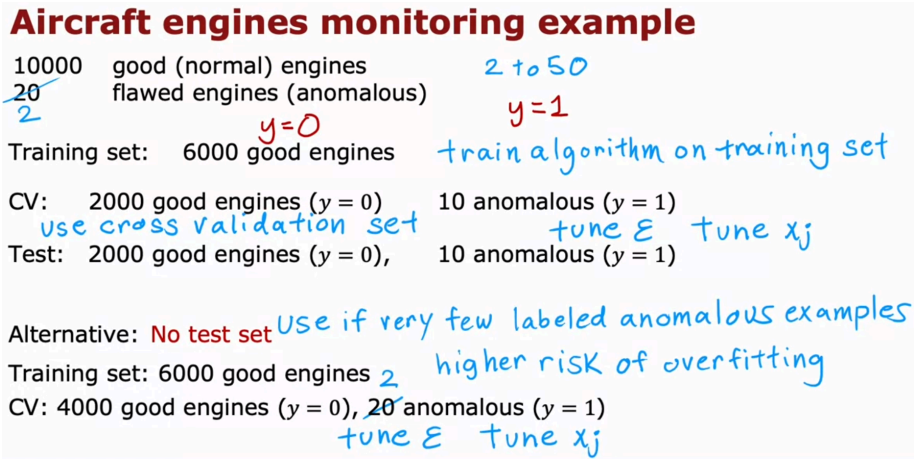
   $p(x) = \prod_{j=1}^{n} p(x_j; \mu_j, \sigma_j^2) = \prod_{j=1}^{n} \frac{1}{\sqrt{2\pi}\sigma_j} exp(-\frac{(x_j - \mu_j)^2}{2\sigma_j^2})$
4. Anomaly if $p(x) < \epsilon$



1.10. Developing and evaluating an anomaly detection system

- *Real-number evaluation:* when developing a learning algorithm (choosing features, etc), making decisions is much easier if we have a way of evaluating our learning algorithm

- Assume we have some labeled data, of anomalous (y=1) and non-anomalous (y=0) examples
- Training set: $x^{(1)}, x^{(2)}, \ldots, x^{(m)}$ (assume normal examples/not anomalous)
- Cross-validation set: $(x_{cv}^{(1)}, y_{cv}^{(1)}), \ldots, (x_{cv}^{(m_{cv})}, y_{cv}^{(m_{cv})})$
- Test set: $(x_{test}^{(1)}, y_{test}^{(1)}), \ldots, (x_{test}^{(m_{test})}, y_{test}^{(m_{test})})$
- Include a few anomalous examples (y=1), mostly normal examples (y=0) in CV and test sets



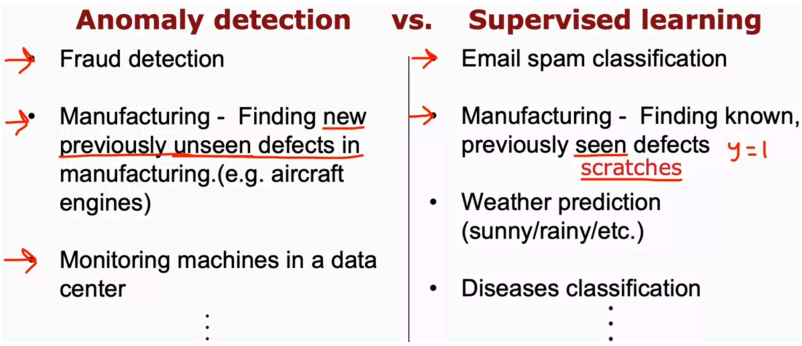
- Fit model $p(x)$ on the training set. Then, on a cross validation/test example, x, predict:

$$y = \begin{cases} 1 & if\ p(x) < \varepsilon\ (anomaly) \\ 0 & if\ p(x) \geq \varepsilon\ (normal) \end{cases}$$

- Possible evaluation metrics include:
  - TP, FP, FN, TN
  - Precision/Recall
  - F1-score
- Use cross validation set to choose parameter ϵ

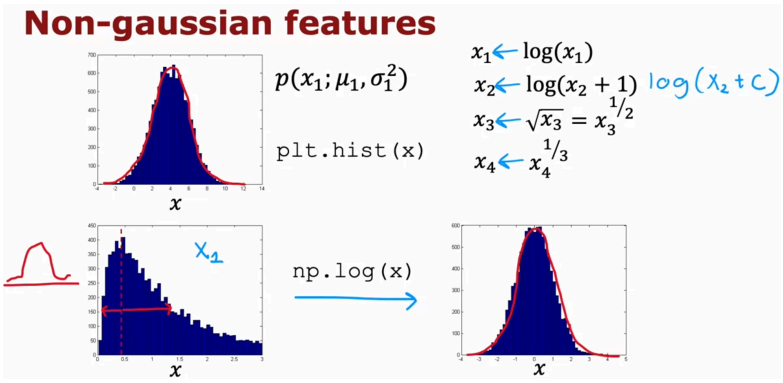
1.11. Anomaly detection vs. supervised learning

- Choose anomaly detection when:
  - Very small number of positive examples (y=1, 0-20 is common)
  - Large number of negative (y=0) examples
  - Many different "types" of anomalies, then it is hard for any algorithm to learn from positive examples what the anomalies look like; future anomalies may look nothing like any of the anomalous examples we've seen so far
- Choose supervised learning when:
  - Large number of positive and negative examples. Might still be okay with a small number of positive examples.
  - Enough positive examples for algorithm to get a sense of what positive examples are like, future positive examples likely to be similar to ones in training set



1.12. Choosing what features to use

- Harder for anomaly detection systems to find what features to ignore
- Try to use (approximately) Gaussian features:



- Want $p(x) \geq \epsilon$ large for normal examples x & $p(x) < \epsilon$ small for anomalous examples x
- Most common problem: $p(x)$ is comparable for normal and anomalous examples
    - See if you can find new features that make it obvious what anomalous examples are (choose features that might take on unusually large or small values in the event of an anomaly)

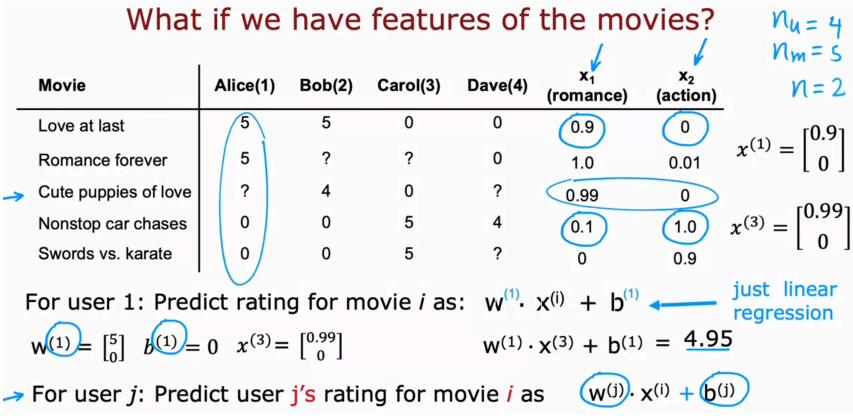
2. Week 2 Notes – Recommender Systems

2.1. Making recommendations

- Recommendations of things you may want to buy or try out
- A typical recommender system has some number of users and some number of items you want to recommend to the users
- n_u is the number of users, n_m is the number of movies (items), $r_{i,j} = 1$ if user j has rated movie i, $y^{(i,j)}$ is the rating given by user j to movie i (defined only if $r(i,j) = 1$)
- Can try to predict which movies users would like, and thus recommend those they would like. Make use of extra features (if available)

2.2. Using per-item features

- Each item has features (like romance, or action features for movies)
- n is the number of features



- Cost function with regularization over movies that the user HAS ALREADY rated:

$r(i,j) = 1$ if user j has rated movie i (0 otherwise)
$y^{(i,j)}$ = rating given by user j on movie i (if defined)
$w^{(j)}, b^{(j)}$ = parameters for user j
$x^{(i)}$ = feature vector for movie i

For user j and movie i, predict rating: $w^{(j)} \cdot x^{(i)} + b^{(j)}$
$m^{(j)}$ = no. of movies rated by user j
To learn $w^{(j)}, b^{(j)}$

$$min_{w^{(j)}b^{(j)}} J(w^{(j)}, b^{(j)}) = \frac{1}{2} \sum_{i:r(i,j)=1} (w^{(j)} \cdot x^{(i)} + b^{(j)} - y^{(i,j)})^2 + \frac{\lambda}{2} \sum_{k=1}^{n} (w_k^{(j)})^2$$

- Note the elimination of $m^{(j)}$ in the $\frac{1}{2m^{(j)}}$ and $\frac{\lambda}{2m^{(j)}}$ term. This is commonly done for recommender systems.
- This cost function is for a single user. To learn parameters for all users, $w^{(1)}, b^{(1)}, w^{(2)}, b^{(2)}, \ldots, w^{(n_u)}, b^{(n_u)}$:

$$J(w^{(1)}, \ldots, w^{n_u}, b^{(1)}, \ldots, b^{(n_u)}) = \frac{1}{2} \sum_{j=1}^{n_u} \sum_{i:r(i,j)=1} (w^{(j)} \cdot x^{(i)} + b^{(j)} - y^{(i,j)})^2 + \frac{\lambda}{2} \sum_{j=1}^{n_u} \sum_{k=1}^{n} (w_k^{(j)})^2$$

- Use gradient descent or other optimization algorithms to find your set of parameters for all the users. Very similar to linear regression.

2.3. Collaborative filtering algorithm

- Per-item features can be used to learn parameters with the above cost function, similar to linear regression. If you don't have those features, you can come up with them from the data.
- If you already have the parameters (through a method learned later), you can work backwards to find your features (system of equations or linear algebra)

- Given $w^{(1)}$, $b^{(1)}$, $w^{(2)}$, $b^{(2)}$, …, $w^{(n_u)}$, $b^{(n_u)}$, to learn $x^{(i)}$:

  $$min_{x^{(i)}} J(x^{(i)}) = \frac{1}{2} \sum_{j:r(i,j)=1} (w^{(j)} \cdot x^{(i)} + b^{(j)} - y^{(i,j)})^2 + \frac{\lambda}{2} \sum_{k=1}^{n} (x_k^{(i)})^2$$

- To learn $x^{(1)}$, $x^{(2)}$, …, $x^{(n_m)}$:

  $$J(x^{(1)}, x^{(2)}, \ldots, x^{(n_m)}) = \frac{1}{2} \sum_{i=1}^{n_m} \sum_{j:r(i,j)=1} (w^{(j)} \cdot x^{(i)} + b^{(j)} - y^{(i,j)})^2 + \frac{\lambda}{2} \sum_{i=1}^{n_m} \sum_{k=1}^{n} (x_k^{(i)})^2$$

- Collaborative filtering puts together the cost functions for finding parameters and finding per-item features:

  $$J(w, b, x) = \frac{1}{2} \sum_{(i,j):r(i,j)=1} (w^{(j)} \cdot x^{(i)} + b^{(j)} - y^{(i,j)})^2 + \frac{\lambda}{2} \sum_{j=1}^{n_u} \sum_{k=1}^{n} (w_k^{(j)})^2 + \frac{\lambda}{2} \sum_{i=1}^{n_m} \sum_{k=1}^{n} (x_k^{(i)})^2$$

- Can use gradient descent for this cost function to find $w, b, and\ x$:

repeat {

~~$w_i = w_i - \alpha \frac{\partial}{\partial w_i} J(w,b)$~~

~~$b = b - \alpha \frac{\partial}{\partial b} J(w,b)$~~

$$w_i^{(j)} = w_i^{(j)} - \alpha \frac{\partial}{\partial w_i^{(j)}} J(w,b,x)$$

$$b^{(j)} = b^{(j)} - \alpha \frac{\partial}{\partial b^{(j)}} J(w,b,x)$$

$$x_k^{(i)} = x_k^{(i)} - \alpha \frac{\partial}{\partial x_k^{(i)}} J(w,b,x)$$

}

2.4. Binary labels: favs, likes, and clicks

- From linear regression-like model to binary classification for binary labels
- Previously, predicted $y^{(i,j)}$ as $w^{(j)} \cdot x^{(i)} + b^{(j)}$
- For binary labels, predict that the probability of $y^{(i,j)} = 1$ is given by:

  $g(w^{(j)} \cdot x^{(i)} + b^{(j)})$, where $g(z) = \frac{1}{1+e^{-z}}$ (like logistic regression!)
- Cost function for binary application switches:

  $y^{(i,j)}$: $f_{(w,b,x)}(x) = g(w^{(j)} \cdot x^{(i)} + b^{(j)})$

  $L(f_{(w,b,x)}(x), y^{(i,j)}) = -y^{(i,j)} log(f_{(w,b,x)}(x)) - (1 - y^{(i,j)}) log(1 - f_{(w,b,x)}(x))$
- That is the loss for a single example (binary cross entropy)

  $$J(w, b, x) = \sum_{(i,j);r(i,j)=1} L(f_{(w,b,x)}(x), y^{(i,j)})$$
- That is the cost function for all examples

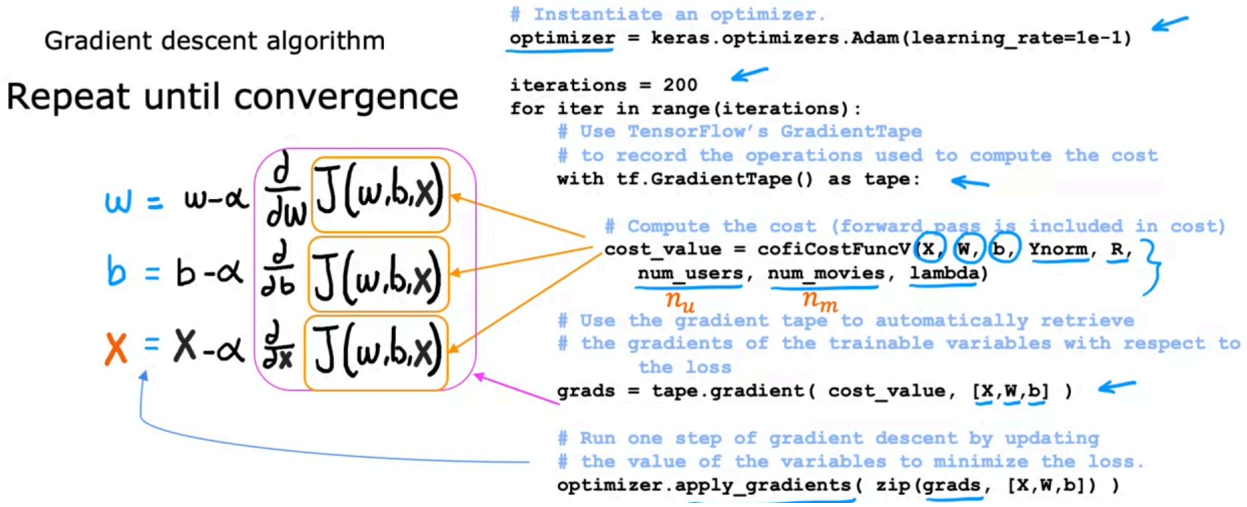
2.5. Mean normalization

- Recommender algorithms run more efficiently and perform a little bit better if you normalize means (have same average value)

- For each item or movie, compute the average rating/value, and create a mean, μ, vector.
- Then subtract the mean from each original value. This gives a new matrix of scores/values that is mean-normalized. This gives a new $y^{(i,j)}$
- On new predictions, use $w^{(j)} \cdot x^{(i)} + b^{(j)} + \mu_i$ (essentially add back the mean for that item).
- This causes new users to have mean ratings for each item by default.
- You can also normalize columns (people rather than items), but it depends on application.

2.6. TensorFlow implementation of collaborative filtering

- TensorFlow uses autodiff for auto differentiation (sometimes called autograd)



2.7. Finding related items

- How do websites give you similar items to the one you are currently looking at? Collaborative filtering!
- The features $x^{(i)}$ of item i are quite hard to interpret in practice (when learned automatically)
- To find other items related to item i, find item k with $x^{(k)}$ similar to $x^{(i)}$

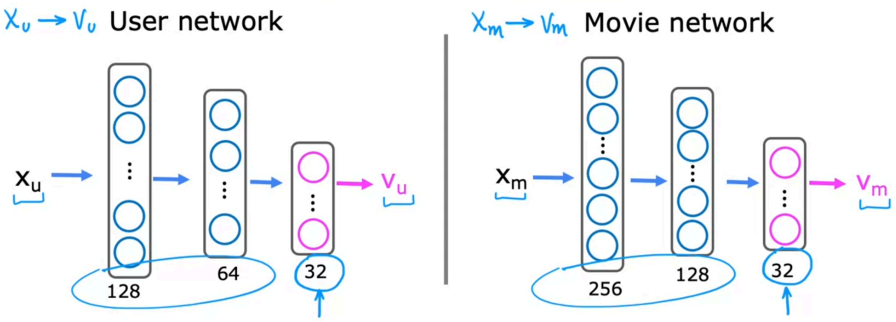
  $\sum_{l=1}^{n} (x_l^{(k)} - x_l^{(i)})^2 = ||x^{(k)} - x^{(i)}||^2$ (i.e., with smallest distance)
- Limitations of collaborative filtering:
  - Cold-start problem:
    - How do you rank a new item that few users have rated?
    - How do we show something reasonable to users who have few ratings?
  - You can use side information about items or users (not used by collab. filtering):
    - Item: genre, movie starts, studio, …
    - User: demographics (age, gender, location), expressed preferences, …

2.8. Collaborative filtering vs. content-based filtering

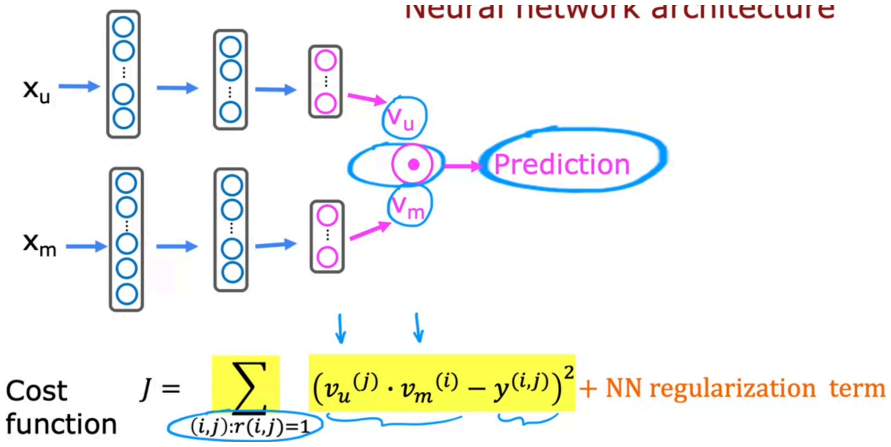
- Collaborative filtering recommends based on ratings of users who gave similar ratings to you
- Content-based filtering recommends based on features of user and item to find a good match
- Examples of user features include: age, gender, country, movies watched, average rating per genre ($x_u^{(j)}$ for user j is the vector of user features)
- Examples of movie features include: year, genre/genres, reviews, average rating ($x_m^{(i)}$ for movie i is the vector of movie features)
- Content-based filtering learns to match users to movies. Bias does not affect this algorithm much (can neglect). Can predict rating of user j on movie i as: $v_u^{(j)} \cdot v_m^{(i)}$; where $v_u^{(j)}$ and $v_m^{(i)}$ are vectors computed from $x_u^{(j)}$ and $x_m^{(i)}$, respectively
- These vectors must be the same size, whereas x feature vectors do not.

2.9. Deep learning for content-based filtering

- Deep learning commonly used for content-based filtering
- User network turns x_u *to* v_u. Movie network turns x_m *to* v_m:



- Can draw them as if it was a single neural network:



- To find movies similar to movie i: $||v_m^{(k)} - v_m^{(i)}||^2$ small
- This allows for pre-computing of similar movies/items, to improve efficiency of recommendations

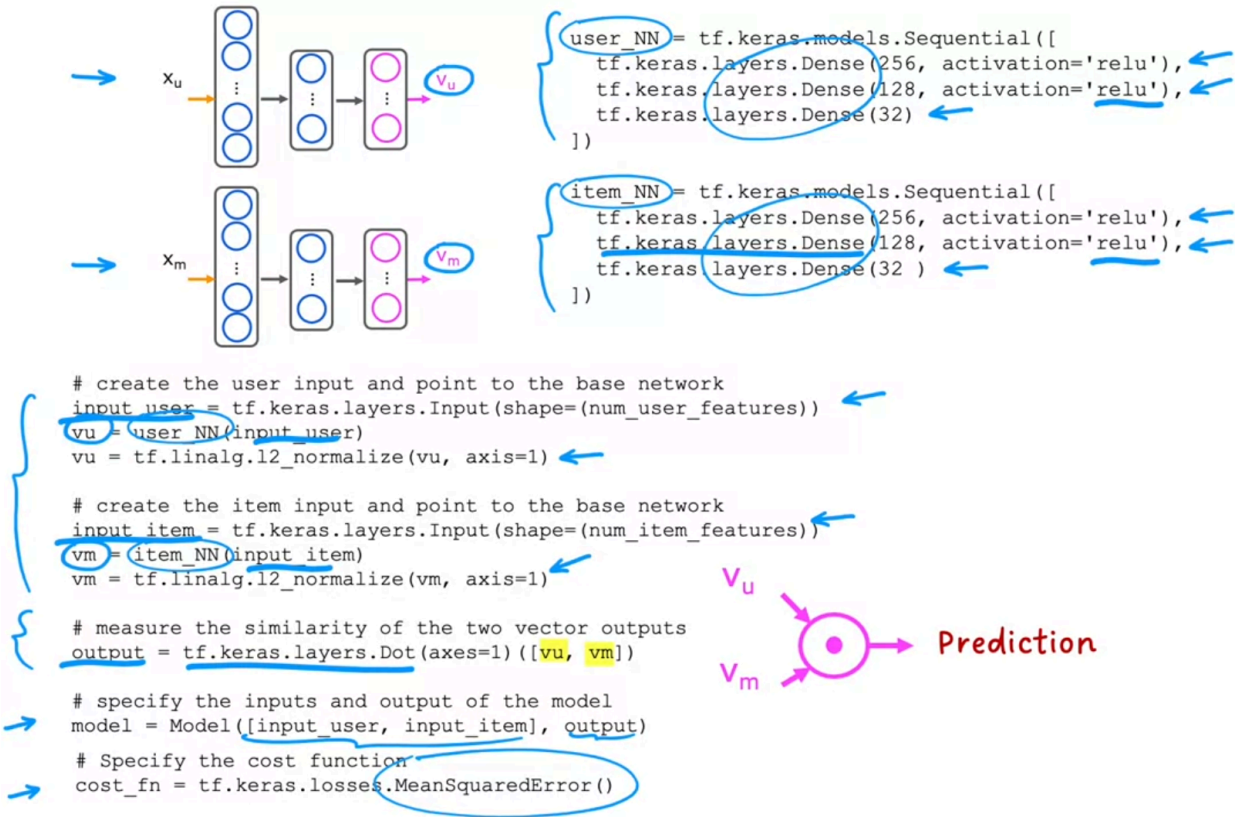
2.10. Recommending from a large catalogue

- Many large-scale recommender systems are implemented in 2 steps:
    1. Retrieval:
        - Generate large list of plausible item candidates
            - E.g. for each of the last 10 movies watched by the user, find 10 most similar movies
            - For most viewed 3 genres, find the top 10 movies
            - Top 20 movies in the country
        - Combine retrieved items into list, removing duplicates and items already
    2. Ranking:
        - Take list retrieved and rank using learned model (feed user and movie feature vectors into neural network and predict rating)
        - Display ranked items to user
- Retrieving more items results in better performance, but slower recommendations
- To analyse/optimize the trade-off, carry out offline experiments to see if retrieving additional items results in more relevant recommendations (i.e., $p(y^{(i,j)}) = 1$ of items displayed to users are higher).

2.11. Ethical use of recommender systems

- The goal is to recommend:
    - Movies most likely to be rated 5 stars by user
    - Products most likely to be purchased
    - Ads most likely to be clicked on
    - Products generating the largest profit
    - Video leading to maximum watch time
- Amelioration: do not accept ads from exploitative businesses

2.12. TensorFlow implementation of content-based filtering



2.13. Reducing the number of features

- If you have a dataset with lots of features, you cannot plot that many features. PCA lets you take those many features, and reduce it to 2-3 for plotting/visualization.
- Example: car measurements include height, length, diameter of wheel, width, etc..
    - Length can change dramatically, but width stays constant as lane width is pre-defined. One way to reduce feature is to keep length, but remove width
    - If you have two features with useful information, you can engineer a third feature that describes both, like “size” instead of width and length
- PCA: find one or more new axes that can represent your data accurately with fewer features
- In practice, PCA is used to reduce 10s-1000s of features down to 2-3

2.14. PCA algorithm

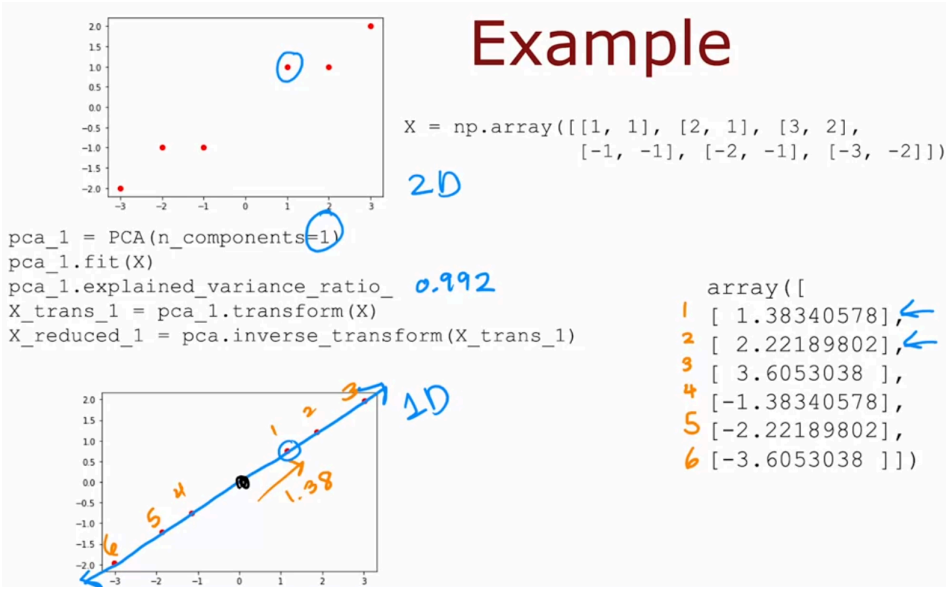
- PCA is unsupervised learning algorithm (no y or labels to learn from)
- Features should be normalized to have zero mean and scaled (to have similar scales)
- Project examples onto the new axis. Greater variance of data points on the new axis is better as it maintains more information from the original data
- This axis is called the principal component: max variance when you project your data onto it

- To find new coordinate on the principal component axis:
    - Length 1 vector is about [0.71, 0.71]
    - If a data point is [2,3], you can take dot product between [2,3] dot [0.71, 0.71]
    - This gives a new coordinate of 3.55 (distance from origin to our projected data point on the new principal component)
- If you find more principal components, they will always be perpendicular to each other
- PCA is not linear regression: you have no labeled data and are not trying to fit a line to predict labels from features.
- Can use reconstruction in PCA to use the transformed data to the original data. Doesn't work exactly, but can provide an approximation 3.55 x [0.71, 0.71] = [2.52, 2.52]

2.15. PCA in code

- Optional pre-processing: perform feature scaling

1. "Fit" the data to obtain 2 (or 3) new axes (principal components)
    a. Also works for more dimensions if needed for application
    b. Use fit function from scikit-learn (automatically does mean normalization)
2. Optionally examine how much variance is explained by each principal component
    a. Use $explained_variance_ratio_$
3. Transform (project) the data onto the new axes
    a. Use $transform$



- Applications of PCA:
    - Visualization (reduce to 2-3 features)
    - Less common:
        - Data compression (reduce storage or transmission costs)
        - Speed up training of supervised training model (features = 1000 → 100)

3. Week 3 Notes – Reinforcement Learning

3.1. What is reinforcement learning?

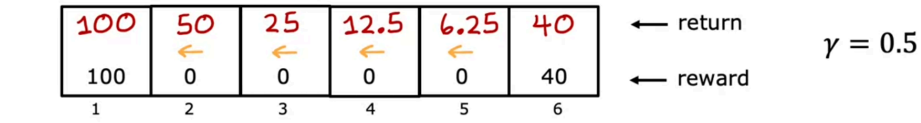
- Learning to fly a helicopter
  - Given state s (position of helicopter), how do we decide action a (how to move control sticks)?
  - Reward function signals when its doing well
    - Positive reward: helicopter flying well (+1)
    - Negative reward: helicopter flying poorly (-1000)
- You have to tell it what to do rather than how to do it, and specify the reward function
- Applications include controlling robots, factory optimization, financial (stock) trading, and playing games (including video games)

3.2. Mars rover example

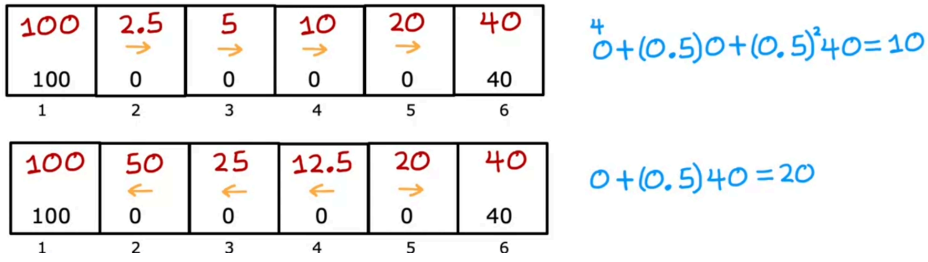
- Position is called state. Assume there are 6 possible states, and the rover starts in state 4.
- Higher reward for the state we prefer
- Terminal state: state where a reward is gathered, but nothing else occurs after this state
- (s, a, R(s), s') → (state, action, reward at state s, new state from taking action s)
  - (4, ←, 0, 3) → (state 4, move left, reward 0 at state 3, state 3 from moving left)

3.3. The return in reinforcement learning

- The return captures the concept that quicker returns may be more worth it than greater rewards much farther away
- Return is sum of rewards weighted by the *discount factor*:
  - $R_1 + \gamma R_2 + \gamma^2 R_3 + \ldots + terminal\ state$; $\gamma = 0.9$ for this example.
  - Makes the algorithm a little bit more impatient as time goes on
  - Common choice is close to 1 (0.9, 0.99, 0.999)
  - We will use discount factor of 0.5 (more impatient) for example
- The return depends on the actions you take



The return depends on the actions you take.



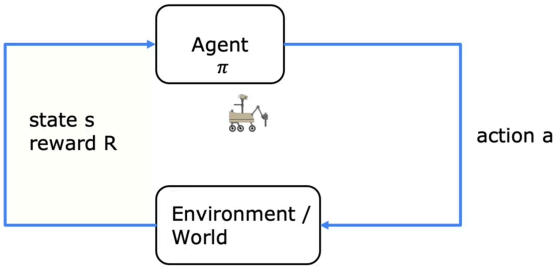
3.4. Making decisions: policies in reinforcement learning

- Many different ways you can take actions in a RL problem
- Our goal is to come up with a function called a policy, π, which takes a state, s, and maps it to an action, a, to maximize the return. $a = \pi(s)$

3.5. Review of key concepts

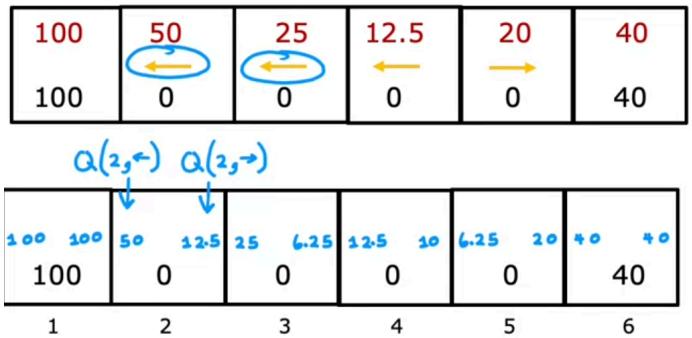
	Mars rover 	Helicopter 	Chess 
states	6 states	position of helicopter	pieces on board
actions	← →	how to move control stick	possible move
rewards	100, 0, 40	+1, −1000	+1, 0, −1
discount factor γ	0.5	0.99	0.995
return	$R_1 + \gamma R_2 + \gamma^2 R_3 + \cdots$	$R_1 + \gamma R_2 + \gamma^2 R_3 + \cdots$	$R_1 + \gamma R_2 + \gamma^2 R_3 + \cdots$
policy π	100 ← ← ← → 40	Find $\pi(s) = a$	Find $\pi(s) = a$

- This is called a *Markov Decision Process (MDP)*. Markov means the future depends only on the current state (not what has happened before)



3.6. State-action value function definition

- State-action value function, $Q(s, a)$, gives a number equal to the return if you start in state s, take action a (once), and behave optimally after that (take actions resulting in highest possible return)



- This is often also called the *Q function*. Tells you how good it is to take action, a, from state, s.
- Gives you a good way to pick actions. Just pick the actions for each state which yields the highest state-action value from the *Q function*. In other words: the best possible return from state s is $max_a Q(s, a)$.
- Q function is also referred to as Q^* or *Optimal Q function*

3.7. State-action value function example

State Action Value Function Example

In this Jupyter notebook, you can modify the mars rover example to see how the values of Q(s,a) will change depending on the rewards and discount factor changing.

```
In [1]: import numpy as np
        from utils import *

In [2]: # Do not modify
        num_states = 6
        num_actions = 2

In [7]: terminal_left_reward = 100
        terminal_right_reward = 40
        each_step_reward = 0

        # Discount factor
        gamma = 0.9

        # Probability of going in the wrong direction
        misstep_prob = 0

In [6]: generate_visualization(terminal_left_reward, terminal_right_reward, each_step_reward, gamma, misstep_prob)
```

Optimal policy

100.0	50.0	25.0	12.5	6.25	10.0
100	← 0	← 0	← 0	← 0	10

Q(s,a)

100.0	100.0	50.0	12.5	25.0	6.25	12.5	3.12	6.25	5.0	10.0	10.0
100		0		0		0		0		10	

3.8. Bellman equation

- The bellman equation helps compute $Q(s, a)$ at every state
- s is the current state, $R(s)$ is the reward of the current state, a is the current action, s' is the state you get to after taking action a, a' is the action that you take in state s'
- The Bellman equation is:
  $Q(s, a) = R(s) + \gamma \cdot max_{a'} Q(s', a')$
- In a terminal state, this simplifies to: $Q(s, a) = R(s)$
- $R(s)$ is the immediate reward, $\gamma \cdot max_{a'}(s', a')$ is the return from behaving optimally starting from state s'

3.9. Random (stochastic) environment

- In practice, noise exists, meaning the action may not be exactly what we expect
- Add a chance (probability) of misstep (90% goes right when commanded to go right, 10% goes left when commanded to go right)
- Maximize the average value of all possible reward sequences. This is called the expected return: $Expected\ Return = Average(R_1 + \gamma R_2 + \gamma^2 R_3 + \ldots)$
- Goal of RL: choose a policy $\pi(s) = a$ that will tell us what action a to take in state s so as to maximize the expected return.
- The modified Bellman equation for stochastic environments is: $Q(s,a) = R(s) + \gamma E[max_{a'} Q(s',a')]$

3.10. Example of continuous state space applications

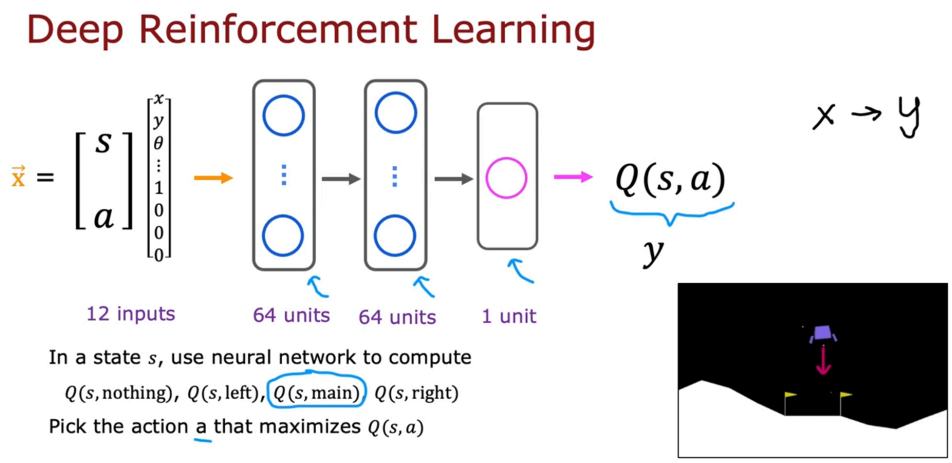
- Most applications have continuous state spaces: not a fixed number of states (discrete), but a continuous number of states
- A truck/car, the state may include: $s = [x, y, \theta, \dot{x}, \dot{y}, \dot{\theta}]$. Any of these numbers may take on any value between its valid range. E.g., $\theta = [0, 360]\ deg$
- For autonomous helicopter: $s = [x, y, z, \phi, \theta, \omega, \dot{x}, \dot{y}, \dot{z}, \dot{\phi}, \dot{\theta}, \dot{\omega}]$

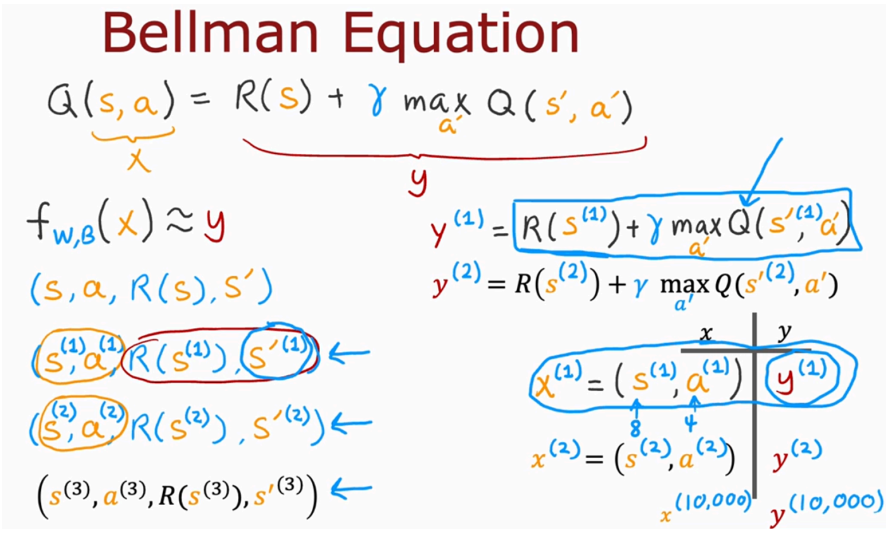
3.11. Lunar lander

- Actions: do nothing, left thruster, main thruster, right thruster
- $s = [x, y, \dot{x}, \dot{y}, \theta, \dot{\theta}, l, r]$. l & r correspond to whether the left and right legs are sitting on the ground (binary values)
- Reward function:
  - Getting to landing pad: 100-140
  - Additional reward for moving toward/away from pad
  - Crash: -100
  - Soft landing: +100
  - Leg grounded: +10
  - Fire main engine: -0.3
  - Fire side thruster: -0.03
- Learn a policy, π, that, given s, picks action $a = \pi(s)$ so as to maximize the return
- $\gamma = 0.985$

3.12. Learning the state-value function

- Use a neural network to approximate the state-action value function, $Q(s,a)$
- This is called deep reinforcement learning
- Action vector is 1-hot encoded to signify which action is taken
- Use Bellman equation to create a training set of x → y

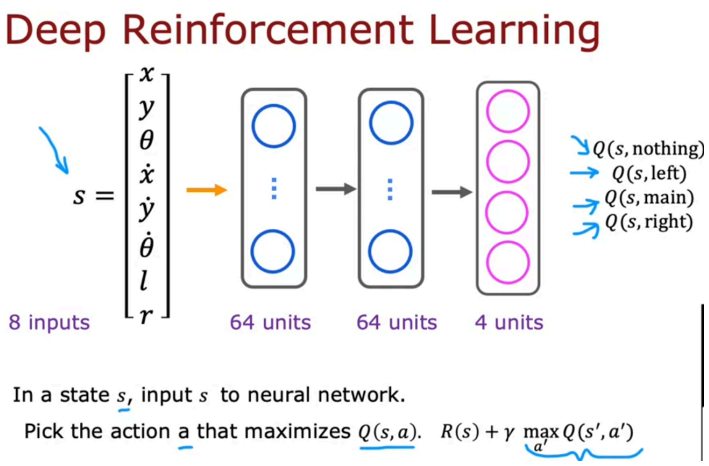




- Initialize neural network randomly as guess of $Q(s,a)$
- Repeat:
  - Take actions in lunar lander, get $(s, a, R(s), s')$
  - Store 10,000 most recent $(s, a, R(s), s')$ tuples (this is called *replay buffer*)
  - Train neural network:
    - Create training set of 10,000 examples using $x = (s,a)$ and $y = R(s) + \gamma \cdot max_{a'} Q(s',a')$
    - Train Q_{new} such that $Q_{new}(s,a) \approx y$
  - Set $Q = Q_{new}$
- Sometimes called *DQN* (Deep Q-Network) algorithm

3.13. Algorithm refinement: improved neural network architecture

- Carrying out inference 4 times for every state is inefficient. Instead, train a single neural network to output all 4 values simultaneously (or all n states)



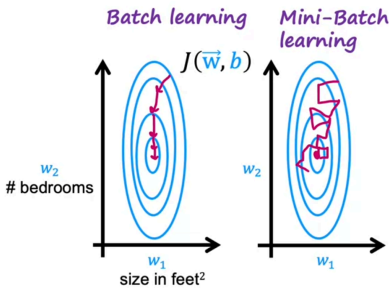
3.14. Algorithm refinement: ε-greedy policy

- How to pick actions while still learning? ϵ-greedy policy
- In some state s:
  - Option 1:
    - Pick the action a that maximizes $Q(s, a)$
  - Option 2:
    - With probability 0.95, pick the action a that maximizes $Q(s, a)$. Greedy, "Exploitation"
    - With probability 0.05, pick an action a randomly. "Exploration"
- Due to random initialization, it may think some good actions are never good ideas. We must build in some random probability of trying out random actions, so that the NN can overcome its preconceptions of a bad idea. This is called an *exploration step.*
- ϵ-greedy policy ($\epsilon = 0.05$)
- A trick is to start ϵ high and gradually decrease so that as you learn, you are less likely to take random actions and you trust your model more (ϵ: $1.0 \rightarrow 0.01$)

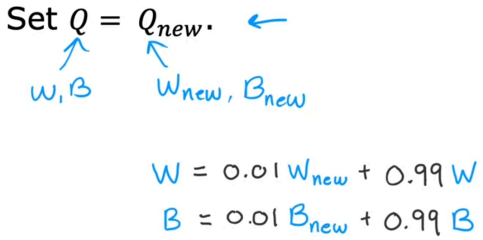
3.15. Algorithm refinement: mini-batch and soft updates

- Say training examples, m, is around 100M. Then, every step of optimization (gradient descent) requires computing average over all 100M examples, which is slow. Instead, use mini-batching, with m', mini-batch size, of 1000, which picks a subset of all examples and computes gradient updates over that smaller subset.

x	y
2104	400
1416	232
1534	315
852	178
...	...
3210	870



- For DQN algorithm, you can take a subset of the stored tuples (replay buffer) and use that subset for each training iteration
- Instead of $Q = Q_{new}$, which can make very abrupt changes (and changes that can lead to worse Q-functions), we can use the soft update method. Make it use 99% of the old model and 1% of the new model parameters (can change the split, hyperparameters).



3.16. The state of reinforcement learning

- Much easier to get an RL algorithm to work in simulation than in a real robot due to stochastic and noisy environments
- Far fewer applications than supervised & unsupervised learning
- Exciting research direction with potential for future applications